PI Frutas — Backend

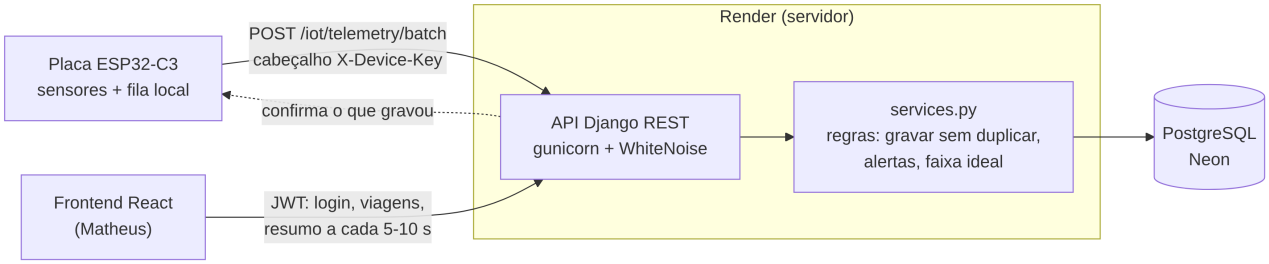
O que foi feito, como foi feito e o que falta (Wendell e Lucas) · 31/10/2026

1. Resumo

O backend é uma API REST em **Django REST Framework**, com banco **PostgreSQL no Neon**, publicada no **Render** em https://projeto-integrador-time1.onrender.com. Ele cadastra as viagens de frutas, recebe da placa ESP32 as leituras de temperatura, umidade e impacto, gera alertas quando a carga sai da faixa ideal e entrega ao front os dados do dashboard em tempo real.

Indicador	Valor
Tabelas do projeto	11 (mais as tabelas de usuários e permissões do Django)
Rotas da API documentadas no Swagger	60 operações
Testes automatizados	71 (autenticação, permissões, placa, alertas, impactos, viagem, seed)
Contrato	Rotas, campos e JSON em inglês, no formato do plano técnico ({"data": ...})

2. Arquitetura



A placa e o front falam só com a API. A API passa toda gravação pelos services, que aplicam as regras e gravam no PostgreSQL. A placa só apaga da memória o que a API confirmou.

Por que essas tecnologias: Django REST Framework entrega autenticação, permissões, validação, paginação e documentação automática prontas. PostgreSQL garante regras no próprio banco (chaves únicas, CHECKs, índices parciais). Neon e Render têm plano gratuito, HTTPS automático e não dependem de um PC ligado.

3. Estrutura do código

O projeto Django **piADS04** é dividido em apps, cada um com uma responsabilidade:

App	Responsabilidade	Principais models
usuarios	Login JWT por e-mail, /auth/me, permissões, criação dos grupos	User, Group, Permission (Django)
dominio	Cadastros de referência e a seed de demonstração	Location, Vehicle, Fruit, FruitParameter
transporte	Viagem: ciclo de vida, faixa ideal da carga, resumo do dashboard	Shipment, ShipmentItem
iot	Placas, credencial, recepção dos dados da placa	Device, DeviceCredential, Reading, ShockEvent
alertas	Motor de alertas de temperatura, umidade e impacto	Alert

App	Responsabilidade	Principais models
piADS04	Configuração, rotas gerais, formato das respostas, página inicial	—

Dentro de cada app, as camadas são sempre as mesmas: **models.py** (tabelas e regras do banco), **services.py** (regras de negócio em Python puro, sem HTTP), **serializers.py** (converte para JSON e valida o que chega), **views.py** (recebe a requisição e chama o service), **urls.py** (rotas) e **tests.py**. A regra é que a view não decide nada: assim a mesma regra serve para a placa, para um futuro MQTT e para a seed.

4. Banco de dados

O banco é gerado pelas **migrations** do Django a partir dos models, nunca à mão no pgAdmin: assim a estrutura fica versionada no Git e igual em todas as máquinas. A viagem (shipments) é o centro: leituras, impactos, carga e alertas apontam para ela.

O DER completo, com todas as colunas e cardinalidades, está no **Anexo** (última página) e em alta resolução no repositório, em docs/DER.png.

Regras que o próprio banco garante

Regra	Como	Por quê
Reenvio da placa não duplica	UNIQUE(device_id, sequence) em readings e UNIQUE(device_id, event_id) em shock_events	A placa reenvia o que não foi confirmado; o banco descarta o repetido
Uma placa em uma viagem ativa por vez	Índice único parcial em shipments (status = active)	Senão não se sabe de qual viagem é a leitura; vale até com dois cliques ao mesmo tempo
Um alerta ativo por viagem e tipo	Índice único parcial em alerts	Evita centenas de alertas iguais
Um limite ativo por fruta	Índice único parcial em fruit_parameters	Sem ambiguidade de qual faixa vale
Valores impossíveis recusados	CHECK: umidade 0–100, temperatura −40 a 80 °C, impacto ≥ 0, mínimo < máximo, origem ≠ destino, fim depois do início	Vale para qualquer caminho de gravação, não só a API

Correções do DER anterior

No DER do brModelo, as cardinalidades dos relacionamentos 1:N estavam do lado trocado (por exemplo, "medição (1,1)" dizia que cada alocação tinha uma única medição). O documento **docs/MODELO_DE_DADOS.md** lista as 7 correções e a correspondência de cada entidade antiga com o banco atual: usuário/perfil/permissão viraram o sistema nativo do Django, alocação virou viagem, e o limite passou de "por dispositivo" para "por fruta", como pede o plano.

5. Como cada parte funciona

5.1 Login, perfis e permissões

- **Login por e-mail** em POST /auth/login. Devolve dois tokens JWT (access de 30 min e refresh de 1 dia) e o usuário com o papel (role: admin, operator ou viewer). O front manda o access em toda chamada (Authorization: Bearer).
- **Senhas** ficam só como hash no banco (sistema nativo do Django, com validadores de senha fraca).

- **Perfis** são grupos do Django criados pelo comando criar_grupos: Administrador (tudo), Operador (viagens, cargas, locais, veículos e alertas) e Visualizador (só leitura).
- **Permissão**: toda rota exige login e a permissão do grupo, inclusive para ler. Ações como iniciar viagem exigem permissão de alteração.
- **Força bruta**: no máximo 10 tentativas de login por minuto por IP (resposta 429 depois disso).

5.2 Viagem

- Ciclo: **planned → active → completed** (ou cancelled), pelas rotas /start e /finish. O status não pode ser editado direto. Quem cria a viagem fica registrado (created_by).
- **Carga mista:** uma viagem pode levar várias frutas. A faixa ideal é a interseção das faixas (maior mínimo e menor máximo), porque o caminhão tem uma temperatura só; a margem e o impacto máximo usados são os mais rigorosos. Fruta sem faixa em comum com as outras é recusada.

5.3 Recepção dos dados da placa

A placa envia lotes para **POST /api/v1/iot/telemetry/batch**:

```
{"device_uid": "VSF-001", "shipment_id": 3, "network": "wifi",
 "readings": [{"sequence": 1001, "recorded_at": "2026-10-31T10:00:00Z",
               "temperature_c": 7.8, "humidity_pct": 88.2, "impact_peak_g": 1.6}],
 "shocks":   [{"event_id": "evt-001", "recorded_at": "2026-10-31T10:02:00Z", "impact_peak_g": 3.1}]}

Resposta: {"data": {"accepted_reading_sequences": [1001], "accepted_shock_ids": ["evt-001"]}}
```

Passo	O que acontece	Por quê
1. Autenticar a placa	A chave vem no cabeçalho X-Device-Key. O banco guarda só o hash SHA-256 dela; a chave aparece uma única vez, ao ser gerada.	A placa não usa login de pessoa; chave vazada pode ser revogada
2. Validar	Formato, valores, até 200 itens; device_uid igual ao da chave; a viagem tem que ser desta placa e estar ativa (ou concluída, para dado atrasado).	Uma placa não grava na viagem de outra
3. Ordenar	As leituras são processadas por recorded_at.	A regra das 2 leituras seguidas funciona mesmo com lote atrasado ou embaralhado
4. Gravar item a item	Cada item entra numa transação própria (savepoint). Repetido: o banco recusa e ele conta como já gravado. Valor impossível: recusado e não confirmado.	Um item ruim não derruba o lote
5. Avaliar alertas	Cada leitura nova passa pelo motor de alertas.	—
6. Confirmar	Devolve só os sequence e event_id que estão no banco. Atualiza a última comunicação da placa.	A placa apaga da fila só o confirmado

Sem internet: a placa continua medindo e guardando na fila local; quando a rede volta, reenvia tudo. As leituras entram com o horário original (recorded_at), e o horário de chegada (received_at) mostra o que foi sincronizado depois. A leitura é ligada à viagem por chave, então um relógio errado não a tira da viagem.

Também existem **/iot/heartbeat** (a placa avisa que está viva e informa a rede Wi-Fi) e **/iot/config** (a placa pergunta qual é sua viagem ativa e os limites).

5.4 Alertas

Situação	O que o sistema faz
Dentro da faixa	Normal. Se havia alerta ativo daquele tipo, ele é resolvido sozinho.
Fora da faixa até a margem crítica	Nível warning (atenção). Ex.: manga 10–13 °C com margem 3: 14 °C.
Além da margem crítica	Nível critical. Ex.: 16,5 °C.
Primeira leitura fora	Ainda não alerta: precisa de 2 leituras seguidas fora (o DHT11 erra cerca de 2 °C e 5%; evita alarme falso).
Continua fora	Não cria outro alerta; só sobe para crítico se piorar.
Impacto acima do limite da fruta	Alerta na hora (é um evento pontual). Vários impactos atualizam o mesmo alerta com o maior pico; acima de 1,5× o limite é crítico. Encerrado por uma pessoa (/alerts/{id}/resolve).

Cada alerta guarda o valor medido e o limite violado **no momento**, uma mensagem pronta para o front e quem reconheceu. A regra roda também para dado atrasado de viagem já concluída.

5.5 Dashboard e tempo real

GET /shipments/{id}/summary devolve, numa chamada: última leitura, mínimo, máximo e média, impactos, maior pico, leituras sincronizadas após offline, situação da placa (online/sem comunicação, última comunicação, rede), limites da carga, alertas abertos e o status atual (no_data, normal, warning, critical). O front atualiza a cada 5–10 s e busca só as leituras novas com /readings?since=. O plano aceita esse modelo (polling) sem WebSocket.

5.6 Formato da API

- Toda resposta de sucesso vem em {"data": ...}; listas trazem "pagination" (next, previous, count). Erros vêm no padrão {"detail": ...}. Isso é feito uma vez só, num renderer e numa paginação próprios (piADS04/api.py).
- Rotas e campos em inglês, como no plano (/fruits, /shipments, /devices, /alerts), aceitando com ou sem barra.
- Documentação automática no Swagger (/api/docs/), incluindo a autenticação da placa.
- A raiz do site mostra que a API está online; rota inexistente responde 404 em JSON.

6. Deploy e configuração

Item	Como foi feito
Servidor	Render (web service gratuito) rodando gunicorn; WhiteNoise serve os arquivos do admin e do Swagger
Banco	PostgreSQL no Neon, conexão com SSL obrigatório (DB_SSLMODE=require)
Segredos	Nada sensível no código: SECRET_KEY, dados do banco e endereços vêm de variáveis de ambiente (.env local, painel do Render). O .env não vai para o Git
A cada deploy	build.sh instala dependências, junta os arquivos estáticos, roda as migrations e recria os grupos
Endereço	O código lê o endereço que o Render informa (RENDER_EXTERNAL_HOSTNAME), evitando o erro 400
Dados de demonstração	python manage.py seed: 3 usuários, 5 frutas, 2 placas com chave, 3 viagens, leituras e alertas gerados pelas mesmas regras da placa

7. Segurança

Medida	Situação
Senhas com hash; JWT com access curto (30 min)	Feito
Toda rota exige login e permissão do grupo, inclusive leitura	Feito
Placa com credencial própria (só o hash no banco), revogável	Feito
Limite de tentativas de login e de envios por placa; lote com no máximo 200 itens	Feito
Regras e valores garantidos no banco (UNIQUE, CHECK, índices parciais)	Feito
HTTPS (Render) e SSL no banco (Neon); segredos fora do código; DEBUG desligado em produção	Feito
Swagger e admin só para staff em produção	Feito
Permissão por viagem (cada pessoa vê só as suas)	Falta — W2 (Wendell)
Logout que invalida o token; recuperação de senha	Falta — L1 (Lucas)
Cabeçalhos de produção (HSTS, cookies seguros) e caminho do admin no .env	Falta — L4 (Lucas)

8. Testes

São 71 testes automatizados (python manage.py test), rodados no PostgreSQL. Cobrem, entre outros: login por e-mail e limite de tentativas; permissões de cada grupo; lote válido, reenvio sem duplicar, lote fora de ordem, chave errada ou revogada, placa tentando gravar na viagem de outra, valor impossível, lote grande demais; regras de alerta (tolerância, sem duplicar, escalonar, resolver, umidade, impacto); viagem (início, fim, placa em duas viagens); formato {"data"} e paginação; e a seed completa.

9. O que falta

Wendell

Tarefa	Como será feita
W2 · Perfis e permissão por viagem (prioridade)	Grupos Produtor, Motorista e Gerente; campos driver e recipient na viagem (created_by faz o papel de produtor); uma função viagens_visiveis(usuario) usada em todas as consultas, para cada pessoa ver só as viagens em que participa (quem não participa recebe 404). Testes de um perfil tentando acessar a viagem de outro.
W3 · Limites congelados e criação em uma chamada	Copiar a faixa ideal para colunas da viagem no /start e usar essa cópia nos alertas; POST /shipments aceitando fruta, lote e quantidade junto.
W7 · Consumidor MQTT (se a placa usar MQTT)	Comando consumir_mqtt com paho-mqtt: assina frutas/+/telemetria, chama o mesmo process_batch do HTTP e publica a confirmação em frutas/<uid>/ack. Roda no PC na apresentação (o Render gratuito não aceita processo contínuo).
W9 · Functions e triggers	Migration com RunSQL: fn_update_timestamp + triggers de updated_at, e fn_shipment_statistics usada pelo resumo. Os alertas ficam em Python de propósito (testáveis).
Vibração	Campo vibration_rms_g na leitura e alerta após 2 leituras seguidas acima do limite; depende da placa calcular o RMS.

Tarefa	Como será feita
Integração com a placa real	Passar URL, chave e formato do lote ao time do firmware e fazer o teste com Wi-Fi desligado/religado, reenvio e reinício da placa.

Lucas

Tarefa	Como será feita
L1 · Logout e recuperação de senha	Ativar a blacklist do SimpleJWT e criar POST /auth/logout; ligar a rotação do refresh token; criar POST /auth/password-reset (sempre responde 200, para não revelar se o e-mail existe) e POST /auth/password-reset/confirm com o gerador de tokens do Django. O e-mail sai por um serviço com API HTTPS (Resend ou SendGrid), porque o Render gratuito bloqueia SMTP.
L2 · Finalização com vistoria e fotos	App vistoria com InspecaoFinal (uma por viagem: condição good, minor_damage ou damaged e observações) e FotoInspecao; POST /shipments/{id}/finish em multipart; só o destinatário finaliza (depende da W2); fotos até 5 MB, só JPEG/PNG/WebP conferidos pelo Pillow; guardadas num armazenamento externo (o disco do Render é apagado a cada deploy) e servidas só para quem pode ver a viagem.
L4 · Configuração de segurança (restante)	Com DEBUG=False: SECURE_SSL_REDIRECT, cookies seguros e HSTS; caminho do admin vindo do .env; no repositório da placa, tirar o wifi_config.py do Git e trocar a chave do ThingSpeak.
L5 · Simulador da placa	Script que envia lotes no formato do contrato com modos offline, reenvio, temperatura fora da faixa e impacto: plano B para a apresentação se a placa ou o Wi-Fi falharem.

Já adiantados nesta versão (estavam na lista do Lucas): status de rede da placa (online, última comunicação, rede e SSID), limite de tentativas de login, a seed de demonstração e os testes do app dominio.

10. Perguntas prováveis e respostas

Por que usar os usuários do Django e não as tabelas usuario/perfil/permissao do DER?

Porque o Django já traz essas três tabelas (auth_user, auth_group, auth_permission e as tabelas N:N), com senha em hash, validação de senha, sessão e permissões testadas. Refazer isso seria mais código e mais risco de segurança, sem ganho.

Por que o limite é por fruta, e não por dispositivo como no DER antigo?

Porque a faixa ideal depende do que está sendo transportado: a mesma placa leva manga hoje e uva amanhã. O plano técnico define fruit_parameters por fruta.

Como vocês evitam dados duplicados quando a placa reenvia?

Cada leitura tem um contador (sequence) que a placa guarda na memória. O banco tem uma regra única (device_id, sequence): a repetida é recusada pelo próprio banco e a API confirma de novo que ela já existe.

E se a placa ficar sem internet no meio da viagem?

Ela continua medindo e guarda numa fila. Quando a rede volta, envia em lote. A API aceita, mantém o horário original, não duplica, gera os alertas e o resumo mostra quantas leituras chegaram atrasadas.

Como a placa se autentica? Não usa login de usuário?

Não. Cada placa tem uma chave própria, enviada no cabeçalho X-Device-Key. O banco guarda só o hash; a chave pode ser trocada e a antiga deixa de funcionar na hora.

Por que os alertas são em Python e não em trigger no banco, como sugere o plano?

O plano também diz que regra de negócio e autorização ficam no Django. Em Python a regra é testável (há testes para cada caso). No banco ficam as regras de integridade, e as functions e triggers de apoio entram na W9.

Por que esperar duas leituras para alertar?

O DHT11 erra cerca de 2 °C e 5% de umidade. Uma leitura isolada fora da faixa pode ser erro do sensor; duas seguidas indicam um problema real. O número é configurável.

Funciona com mais de uma placa?

Sim. Cada placa tem identificador e chave próprios, a regra de duplicação é por placa e o banco impede uma placa em duas viagens ativas. A seed já cria duas placas.

Por que Render e Neon?

Plano gratuito, HTTPS automático e sem depender de um computador ligado. Limites conhecidos: o servidor dorme após 15 min sem uso (a primeira chamada demora cerca de 1 min) e o Neon tem 1 GB, suficiente para mais de um ano de leituras de uma placa.

Anexo — DER do banco real

Notação pé de galinha: || exatamente um · |o zero ou um · o{ zero ou muitos. Gire a página para ler.

